



Technical Cheat Sheet & Layman-to-Expert Guide: Mechatronian Hub

This guide is designed for presenters who do not have a deep coding background. It translates complex software architectures into simple, powerful analogies and gives you exact, highly professional scripts to memorize so you can confidently answer any technical query from the jury.



Part 1: The Tech Stack Map (Simple Translations)

| Technical Term | What it is (Layman Terms) | The Analogy (How to explain it) |

| :--- | :--- | :--- |

| **React 19** | The structural engine of the website. | It acts like the building's interior designer, dynamically changing elements of the room (the screen) instantly without rebuilding the entire house (reloading the page). |

| **Vite** | The high-speed compiler/bundler. | The assembly line. Instead of packing files slowly, it packs our entire code into a tiny, hyper-compressed delivery box in milliseconds. |

| **Supabase** | Backend-as-a-Service (BaaS). | The engine room. It handles database storage, security gates, user sign-ups, and chat lines so we don't have to build a custom backend from scratch. |

| **PostgreSQL** | Relational Database (SQL). | An Excel spreadsheet on steroids with strict rules. It connects different sheets (users, listings, chats) with unbreakable lines (foreign keys) so data never corrupts. |

| **Row-Level Security (RLS)** | Database security rules. | The keycard system in a hotel. You can only enter the room (read the data) if your keycard (user token) matches that specific room number. |

| **PWA (Progressive Web App)** | Standard web code disguised as a mobile app. | A wrapper. It lets users "install" our website directly onto their phones and receive native notifications without downloading a heavy app from the Play Store. |

| **Vercel** | Global hosting server. | The high-speed highway network. It copies our app's static files to servers all over the world so the site loads under 0.5 seconds regardless of user location. |

Part 2: The Core Significance (Memorize these "Why" answers)

| **1. Why React 19 + Vite instead of plain HTML/JS?**

- **Simple Significance:** Plain HTML/JS requires a full page refresh every time a user clicks something. In an application with chats, notifications, and application feeds, reloading is terrible UX.
- **The Expert Script to say:**

*"Sir, humne React 19 isliye choose kiya taaki hum **Single Page Application (SPA)**

architecture deliver kar sakein. Vite is absolute state-of-the-art—it provides Hot

Module Replacement (HMR) during build, and packages our assets into optimized

bundles. React 19 provides high performance rendering and hooks that update the UI

dynamically without costly page reloads."*

2. Why Supabase + PostgreSQL instead of Node.js + MongoDB?

- **Simple Significance:** MongoDB stores data in loose folders (documents), which is bad for matching people. PostgreSQL stores data in structured tables. A student has a direct relational line to their team requests, which link directly to team listings. Supabase gives us real-time communication out of the box.
- **The Expert Script to say:**

*"Sir, humara app aek relationship-heavy marketplace hai (students, listings,

requests, chats). PostgreSQL (SQL) is relational by nature, guaranteeing data integrity

through strict foreign key constraints. Supabase acts as a highly robust Backend-as-

a-Service, giving us instant OAuth verification, JWT security sessions, and native

Realtime WebSockets for chats, eliminating weeks of backend boilerplate

development."*

Part 3: Deep Dives into Core Features (How it actually works)



1. User Sign Up, OTP & Verification

- **How it works:** When a student enters their email, Supabase generates an 8-digit random code and securely emails it. When the user enters the code, Supabase verifies the cryptographic token and creates a secure session token (JWT) inside the browser.
- **The Expert Script to say:**

*"Hamara user sign-up flow **Supabase GoTrue Authentication** service use karta

hai. Initial registration pe, database user row inject hoti hai with `email_confirmed =`

`false`. User email pe deep-link/OTP dispatch hota hai. OTP match hone par,

Supabase dynamic JWT signature verify karta hai aur state `email_confirmed = true`

update karke session establish karta hai."*



2. LinkedIn "Verified" Checkmark

- **How it works:** We integrate LinkedIn Login (OAuth 2.0). When a user clicks "Verify LinkedIn", they log in on LinkedIn's official site. LinkedIn sends back a secure token proving they are a real professional. Our database updates the user's `is_verified` column to `true`, which triggers the UI to render the green checkmark badge.
- **The Expert Script to say:**

*"LinkedIn verification is driven by an **OAuth 2.0 OpenID Connect (OIDC)**

authentication flow. User LinkedIn portal se authorization grant karta hai. Callback

route securely auth token exchange karke profile information fetch karta hai. Upon

successful handshake, database profile metadata row update hoti hai (setting

`is_verified = true`), jo UI par green verification checkmark trigger karta hai. It

ensures absolute identity authenticity."*



3. Real-time Team Chat

- **How it works:** Instead of checking the server every second for new messages (polling), we use WebSockets. When a message is sent, the database saves it and instantly pushes a message "delta" (the new message) to the active browser channel of team members.
- **The Expert Script to say:**

*"Real-time chat is powered by **PostgreSQL Replication & WebSockets** via

Supabase Realtime engine. UI aek persistent WebSocket connection maintain karta

hai table listeners par. Jab bhi query table **chats** pe inserts stream karti hai,

database replication engine real-time JSON delta push karta hai and UI instantly state

append karke message render kar deta hai without polling."*



4. PWA (Offline & Installation)

- **How it works:** A background script in the browser called a **Service Worker** caches all critical files (CSS, JS, logos). If internet drops, it intercepts the network request and serves the cached files instantly.
- **The Expert Script to say:**

*"Hamara Progressive Web App **Service Worker caching strategies** leverage karta

hai. `sw.js` registration path shell-assets (HTML, compiled CSS, JS, logo bundles)

local storage cache mein register karta hai. Offline state detection par, requests fetch

callback cache se serve hote hain, giving it native desktop app performance on

mobile."*



5. Vercel SPA Routing Configuration (Recent Bug Fix)

- **How it works:** Because of React SPA routing, all virtual URLs (like `/login`) must map to `/index.html`. Our previous rule was redirecting *everything* to `index.html`, which was rewriting `sitemap.xml` and Google verification HTML files to `index.html`. We fixed this by using an advanced regex negative lookahead in `vercel.json` to exclude these specific files.
- **The Expert Script to say:**

"Sir, since standard SPAs use client-side history routing, Vercel edge rewrite `/(.*)`

rule is used to fallback to `/index.html`. However, this was intercepting static SEO

metadata like `sitemap.xml` and HTML verification tokens. We resolved this by

implementing an advanced **negative lookahead regular expression** in our

`vercel.json` configuration, white-listing assets, sitemaps, robots.txt, and PWAs from

SPA rewrites to ensure they serve directly from the edge CDN filesystem."



Part 4: 15 Toughest Technical Jury Questions & Exact Answers

Q1: What is the benefit of using React 19 over React 18?

- **Answer:** `"React 19 supports cleaner, out-of-the-box form actions, server actions, and improved rendering performance. It handles asynchronous transitions natively, meaning we don't have to write manual loading states (useTransition hook support), making our dashboards exceptionally smooth."`

Q2: What security measures did you take on the database?

- **Answer:** `"Humne PostgreSQL Row-Level Security (RLS) use kiya hai. Default state is 'deny-all'. Users tables tabhi read ya write kar sakte hain jab query metadata authenticated JWT signature se check hoti hai. For example, direct chats read tabhi hote hain jab check verified hota hai ki requested user ID actual team database key mein exists karti hai."`

Q3: How do you prevent SQL injection on your queries?

- **Answer:** "Sir, we don't write raw SQL queries on the client side. We utilize Supabase client API which translates queries into fully parameterized JSON RPC calls processed via PostgREST on the backend. This completely neutralizes any SQL injection threat vector."

Q4: If I open the developer console, can I edit the state and bypass verification?

- **Answer:** "No, sir. Changing state in React DevTools only modifies the local client UI temporarily. Any secure action (like sending message or accessing admin panels) runs through server-side JWT session validation and RLS checks in the database. Client-side state changes are purely visual and cannot compromise database integrity."

Q5: How do you handle cross-origin resource sharing (CORS) issues?

- **Answer:** "Humne CORS headers strictly whitelist kiye hain `vercel.json` backend definitions mein. API endpoints strictly restrict custom calls from unauthorized domains, ensuring requests only execute from our verified domain origins."

Q6: If two users apply to the same team at the exact same millisecond, how does your database handle concurrency?

- **Answer:** `"PostgreSQL relational databases handle this via ACID transactions and row locking. Both insert operations are executed as independent, isolated transactions. The database processes them sequentially at the engine level, creating unique timestamps and index keys cleanly."`

Q7: Why are you using SQLite locally for dashboards but Supabase in production? Isn't that inconsistent?

- **Answer:** `"Our central application is 100% Supabase-driven for production-grade real-time collaboration. The SQLite components represent localized mockups and local developer sandboxing for development speed. Relational schemas are mirrored so migration path is seamless."`

Q8: What database constraints are in place to keep data clean?

- **Answer:** `"We use strict relational constraint models: Unique email domains, NOT NULL fields for crucial listing properties, and foreign keys with ON DELETE CASCADE so that if a user deletes their profile, all their orphaned listings, chats, and requests are cleanly purged from the system by the database itself."`

Q9: How do you manage site speed under heavy assets like hero banners?

- **Answer:** `"All our static assets (like hero.png and logo.png) are pre-compressed and deployed straight to Vercel's global Edge Network CDN. The HTML utilizes preload link attributes so large images render during initial DOM load without blocking script execution."`

Q10: How does your app handle cross-site scripting (XSS) attacks in chat boxes?

- **Answer:** `"React natively sanitizes all state variables before rendering them in the DOM. Any text input inside our chat boxes is escaped by React engine as raw text strings, completely disabling any malicious inline"`